Introducing Quimera: feedback-driven exploit generation for Ethereum smart contracts using LLMs

03 Jul, 2025

Over the last couple of weeks, I've been working on prototyping a new open-source tool called Quimera. Quimera was born out of my deep interest in automatic exploit generation, the recent advancements in LLM reasoning, and, frankly, a lot of free time. The core idea behind Quimera is to use *feedback-driven* exploit generation for Ethereum smart contracts, leveraging LLMs and Foundry traces. Essentially, the tool provides iterative feedback to an LLM to help it construct smart contract exploits in Solidity, mimicking how a real security researcher would approach the task. This feedback is generated from a combination of source code analysis, on-chain data, and Foundry traces.

General Approach and Features

Quimera is designed to target a single, yet critical, type of vulnerability: one that allows an attacker to drain the funds of a target contract (or contracts). The overall process can

be summarized in the following steps:

1. Fetch the target smart contract's source code and generate a prompt that describes the exploit objective (e.g., generate profit after a flash loan is repaid).
2. Ask the LLM to create or improve a Foundry test case that attempts to exploit the contract.
3. Run the test, inspect the transaction trace, and check whether a profit was made.
4. If the exploit succeeds, stop. If not, return to step 2 and provide the LLM with the failed attempt's trace to help refine the next iteration.

The initial prompt provided to the LLM contains the following information:

- The name, address, and chain of the target contract.
- The complete source code, including all dependencies and a flattened interface of the target contract.
- The current values of public and private variables. Public variables can be accessed directly using Solidity, but private ones are more difficult. For those, we use slither-read-storage to retrieve their values.
- Lastly, we inform the LLM that there is a high-severity vulnerability in the contract(s) that allows for draining funds. While this may not necessarily be true, we follow George Costanza's wisdom: "It's not a lie if you believe it.".

In addition, the LLM is granted access to a set of "tools" it can invoke during its reasoning phase (i.e., in "thinking mode"). These tools allow it to gather the information described above for any arbitrary address, enabling the LLM to dynamically expand its set of targets during the exploit generation process.

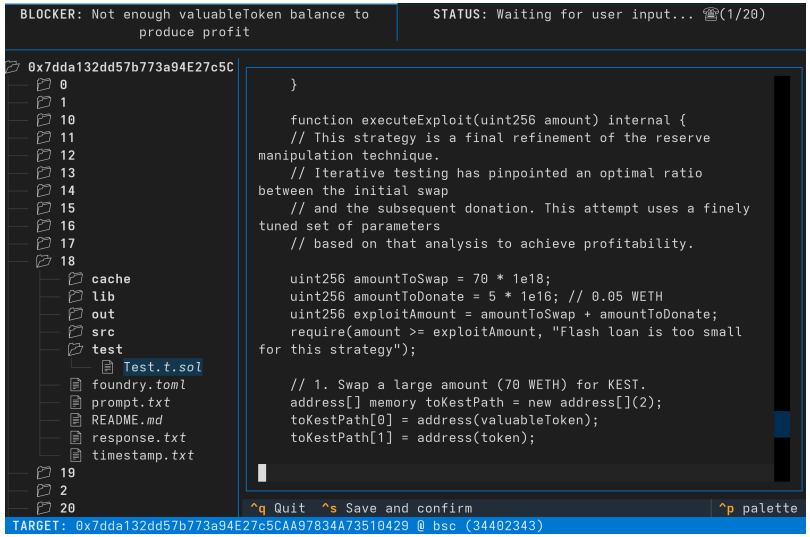
One of the main goals behind Quimera was to explore the practical capabilities of this technology: which LLMs perform better, how to write effective prompts, and what kinds

of exploits are easiest to reproduce. I tested several LLM services (e.g., Gemini, Grok, Claude, DeepSeek) and some local models (e.g., Qwen), and, at the time of writing, the only one I can recommend is Gemini 2.5 Pro. The rest often produce suboptimal results: frequent compiler errors, failure to follow constraints, etc.

On the feature side, I want to highlight two small but useful aspects of Quimera:

Textual User Interface (TUI)

Quimera includes a neat-looking textual user interface (TUI).



This feature isn’t just for aesthetics, it serves a practical purpose. The TUI allows users to **browse previously generated code**, making it easier to manually tweak, analyze, or continue the exploit development without losing track of what the tool has done so far. This is especially useful when an exploit is close to working, *almost* generating profit, but is being blocked by some subtle issue. A human operator can step in, gain quick insights, and push it over the finish line.

Support for In-Development Contracts

Although Quimera was originally built to test already deployed contracts, **it can also be easily used for smart contracts still in development as well**. To do this, you just need to add a new Foundry test that looks like this:

```
...
contract QuimeraBaseTest is Test {
    address public target;
    IWETH public WETH = new MockToken();
    address internal user1 = address(0x123);
    address internal user2 = address(0x456);
    function setUp() public { ... }
    //$executeExploitCode
}
```

In local mode, Quimera will use this test as a starting point. The developer simply needs to set up the contracts and define any relevant constraints to ensure the outcome makes sense. This makes Quimera a flexible tool, not only for reproducing real-world exploits but also for helping developers validate their code during early stages of development.

Positive Results (So Far)

Before diving into some results, it’s important to clarify a methodological concern: closed-source LLMs are trained on massive datasets, which may include known exploits, writeups, or post-mortems of the vulnerabilities we are testing. However, based on my observations, the iterative behavior exhibited during exploit generation doesn’t align

with simple retrieval or "memoized" outputs. Instead, it appears to be a grade of genuine reasoning through each step.

In any case, let's start the positive results, a list of known exploits that I was able to reproduce after a few days of playing with the tool:

Original Exploit	Complexity	Comments
APEMAGA	Low	Only one step needed.
VISOR	Low	A few steps needed to build the WETH conversion calls, but overall the root cause is identified quickly.
FIRE	Medium	It will first build the sequence of calls to exploit it, and then slowly adjust the amounts until profit is found.
XAI	Low	A small number of steps needed.
Thunder-Loan	Low	This one is part of a CTF?

Additionally, after observing the LLM reasoning process for several hours, I've identified three distinct "mental states" that are easy to recognize:

1. Exploration or high-level planning:

This is the phase where the model is trying random ideas just to see what sticks, or where it's attempting to craft a high-level plan. Overall, the model tends to be ineffective here: it often fails to recognize missing steps or logical gaps. I believe this stage could

benefit greatly from the support of a planning agent or an external process (e.g., a fuzzer or static analyzer) to guide the reasoning.

2. Problem-solving or overcoming roadblocks:

This is where the model attempts to fix a specific issue (e.g., a revert caused by an unmet condition). It performs surprisingly well in this stage, likely because the task is concrete and success is easy to measure. The feedback loop is tight, making this a strength for most LLMs I tested.

3. Optimization or fine-tuning a working path:

Once the model finds some code that executes without errors and looks like a potential exploit, it enters a phase where it needs to optimize the result to actually reach the final goal (e.g., achieving profit after repaying a flash loan). The model does an "ok-ish" job here: it tends to iterate slowly by tweaking one parameter at a time and observing the effects. For example, it might adjust a value by 0.01 ETH in each step, when a tenfold increase would be more appropriate. Still, despite the inefficiencies, it usually manages to get the job done eventually.

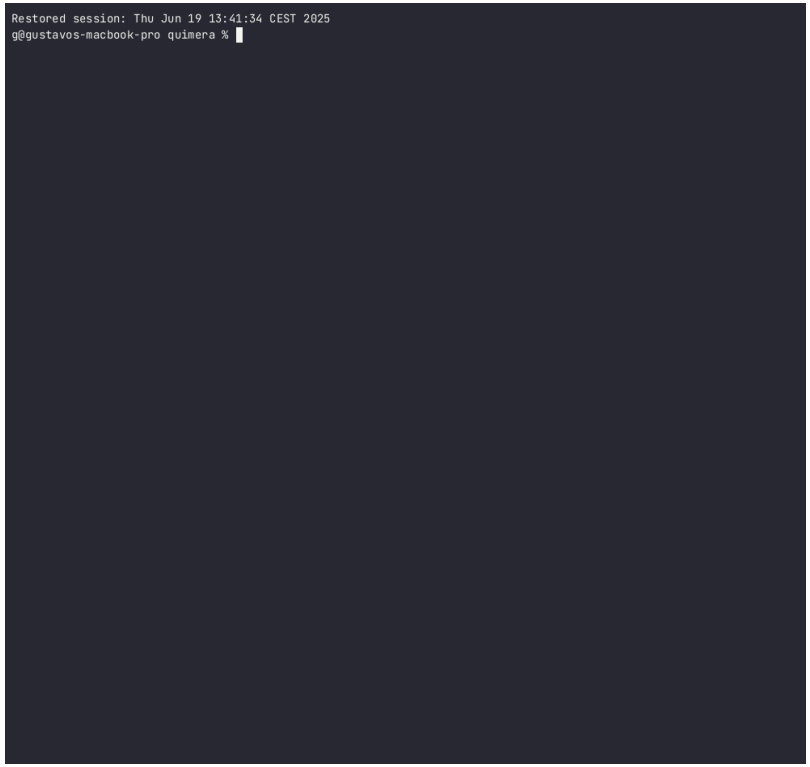
Challenges

I also want to talk about negative results, as they're crucial for understanding the limitations of the current approach, and for identifying how it can be improved.

A specific case worth highlighting is the Alkimiya exploit. This exploit was chosen for reproduction because it strikes a balance: it's neither too easy nor impossibly complex, but it *does* require a very precise sequence of steps to replicate. It's also relatively recent, only a few months old.

In theory, Gemini shouldn't have prior knowledge of this exploit, since its knowledge cutoff is January 2025. Directly asking about it produces hallucinated technical details that are not even remotely close to the real ones, so we're in the clear regarding contamination from training data.

For this small-scale experiment, I ran around 20 iterations in Quimera to see if it could reach the exploit condition. While **the model didn't fully succeed** (i.e., it didn't discover the exploit in a way that results in profit), **it was still able to reproduce most of the steps, including identifying the root cause**. You can see a snapshot of the process here:



(Note: this video is **not** in real-time. It's accelerated to skip the wait time between each response.)

There are several factors that contributed to the failure in producing a working exploit:

This exploit turned out to be **harder than expected**, not just because of the code complexity or the sequence of required function calls, but also because **it doesn't emit strong profit signals during the iterative reproduction process**. The model ends up stuck on a plateau, trying different actions, but none of them lead to any meaningful change in profit. This lack of a signal makes it **very hard to determine a productive direction to explore**. This contrasts with other successful exploit reproductions, where either the profit was easily achieved after a few tweaks, or the model could slowly iterate toward a positive outcome.

That said, Foundry traces still provided useful insights for understanding blockers and dead ends.

Future Work

Besides keep trying to find new exploits to reproduce, I want to test some other ideas:

- Better integration with a planning agent.
- Allow the tool to use fuzzing tools or even symbolic execution in some limited capacity to help it to reach an exploit condition.
- Turn the DefiHack repository into a library of past exploits for Quimera to browse.

As with many trends in software, there's always a chance that this kind of application never moves beyond the proof-of-concept stage. Still, I'm hopeful, and I plan to keep working on it to help uncover what these models are truly capable of.

#Quimera #LLM #Exploit-Generation #Foundry